AutoTraQ stores its data in a **MySQL** database, and the team manages that database using **Prisma**. Prisma works in a "schema-first" way, meaning everything about the database—tables, columns, relationships, and allowed values—is defined in one main file called backend/prisma/schema.prisma. That file is the **single source of truth**: Prisma can use it to create or update the database tables and also generate a **type-safe client** that the backend uses to run database queries safely in TypeScript.

The database is made up of several main tables (called "models" in Prisma). The core ones include **Users** (team members with an email, a hashed password, and a role), **Parts** (the parts catalog with things like SKU, name, condition, cost, and minimum stock), **Inventory Events** (a log of stock moving in or out, including quantity, reason, and which user did it), **Vehicles** (vehicle records like VIN, year, make, and model), and **Requests** (when someone requests parts and the request goes through an approval workflow). There are also supporting tables that add extra features, like **RoleRequests** (users asking for permission upgrades), **PartImages** (photos attached to parts), **VehicleParts** (a join table that connects parts and vehicles in a many-to-many way), **Interchanges** (cross-reference / compatibility info), **PartHierarchy** (categories and subcategories for organizing parts), and **SKUCounter** (used to help auto-generate SKUs per category).

A big reason MySQL works well here is because this project has a lot of relationships. For example, one user can create many inventory events and many requests, one part can have many inventory events and many images, and parts can fit many vehicles (and vehicles can fit many parts), which is why a many-to-many connection table is needed. These relationships keep the data organized and reduce duplication, and foreign keys help ensure the database doesn't end up with "broken" references.

The schema also defines **enums**, which are sets of allowed values that prevent invalid data. For example, a user role must be one of admin, manager, fulfillment, or viewer, and a request status must be something like PENDING, APPROVED, DENIED, FULFILLED, or CANCELLED. This keeps the app consistent, because the database itself enforces what values are allowed.

The typical Prisma workflow is: you define or update models inside schema.prisma, then you sync those changes to the database (commonly with npx prisma db push during development), then you generate the Prisma client (npx prisma generate) so the backend code can query the database using TypeScript-safe methods. The backend services then do things like finding low-stock parts, creating inventory events, or linking parts to vehicles using Prisma queries. There are also seed scripts that populate the database with starter data (like admin accounts, sample parts, vehicles, and SKU counters) so the project is ready for development and demos quickly.

For meeting talking points, you can explain that MySQL was chosen because inventory data naturally has lots of relationships, Prisma makes the schema easy to manage in one place, the database design is normalized to avoid duplication and keep data consistent, enums enforce valid states, the category system is handled with a hierarchy/tree model, inventory movement is tracked using an “append-only” event log for a full audit trail, seed scripts speed up setup, and Prisma’s type-safe client helps catch database query mistakes during development instead of at runtime.